

1. Cronómetro

Construya un programa orientado a objeto que le pida al usuario introducir su nombre en consola, y posteriormente muestre el tiempo que ha demorado en introducirlo siguiendo el formato HH:MM:SS (hora : minutos : segundos). Considere que se cuenta con una clase *Cronómetro*, que presenta la siguiente descripción:

Cronómetro	
Funcionalidad	Descripción
Cronometro()	Construye un cronómetro. Inicialmente está detenido y el tiempo que marca es cero milisegundos.
Arrancar():void	Echa a andar el cronómetro, es decir, comienza a medir el tiempo partiendo de cero.
Detener():void	Detiene el cronómetro. No es posible usar esta funcionalidad sin antes haberse arrancado el cronómetro.
TiempoTranscurrido():int	Retorna el tiempo, en milisegundos, que marca el cronómetro desde que se echó a andar.

2. Automóvil 1

Desarrolle un programa que construya un automóvil a partir de los datos entrados por el usuario, y posteriormente:

- Calcule la cantidad de combustible necesaria para recorrer una cantidad de kilómetros entrada por el usuario.
- Determine si con una cantidad de dinero entrada por el usuario es posible comprar dos automóviles y recorrer 400 Kms en uno de ellos, si la gasolina está a \$ 2.00 el litro.

Automóvil	
Funcionalidades	Descripción
+ Automóvil(tipo: string, fab: string, prec: double, kmXL: int)	Construye un automóvil a partir de un tipo, un nombre de fabricante, el precio de venta, y la cantidad de kilómetros que puede recorrer con un litro de combustible
+ Tipo(): string	Retorna el tipo de automóvil.
+ Fabricante(): string	Retorna el nombre del fabricante.
+ Precio(): double	Retorna el precio de venta.
+ KmPorLitro(): int	Retorna la cantidad de kilómetros que puede recorrer con un litro de combustible.

3. Punto

Dada la clase CPunto descrita a continuación,

+ CPunto
- x: int - y: int
+ CPunto(x1:int,y1: int) + EsDeCuadrante(): int

- **CPunto()**
Construye un punto con la axisa y la ordenada pasadas por parámetro.
- **EsDeCuadrante(): int**
Retorna el número del cuadrante y 0 en caso que esté sobre algún eje.

Desarrolle un programa que:

- Construya dos objetos de la clase CPunto con valores de axisas y ordenadas entrados por el usuario.
- Muestre al usuario un texto que informe en qué cuadrante se encuentra cada punto o si están sobre un eje.

4. Recta

La ecuación cartesiana de una recta se representa $Ax+By+C=0$, donde A, B y C son los coeficientes de la ecuación. Dada la clase Recta descrita a continuación,

- **Recta(a: float, b:float, c:float)**
Construye una recta donde a, b y c son los coeficientes de la ecuación.
- **Pendiente():float**
Retorna el valor de la pendiente de la recta.

Desarrolle un programa que:

- Construya 2 rectas dados sus coeficientes.
- Determine si la recta 1 es paralela con la recta 2. (Dos rectas son $\parallel$ si sus pendientes son iguales)
- Determine si la recta 1 es perpendicular con la recta 2. (Dos rectas son perpendiculares si pendiente de la recta 1 $= -1/\text{pendiente de la recta 2}$)

5. Matriz

Dada la clase CMatriz descrita a continuación,

+ CMatriz
- a: int - b: int - c: int - d: int
+ CMatriz(a:int, b:int, c:int,d: int) + Determinante(): int + EsSingular(): boolean

Desarrolle un programa que:

- Construya una matriz a partir de 4 valores suministrados por el usuario a través del teclado.
- Muestre el determinante de la matriz.
- Muestre un texto donde se informe si la matriz es singular o no.

6. Semáforo

Dada la clase Semáforo descrita a continuación,

+ CSemaforo
- combinacion: Color
+ CSemaforo() + CambiaCombinacion(): void + DiCombinación(): Color

CSemaforo()

Construye un semáforo con la luz en verde.

- **CambiaCombinación():void**
Cambia el color del semáforo según corresponde a las leyes del tránsito(verde, amarillo, rojo, amarillo, verde, etc.)
- **DiCombinación():Color**
Retorna el color actual del semáforo (verde, amarillo o rojo)

Desarrolle un programa que:

- Cree un Semáforo.
- Cambie la combinación de color del semáforo
- Muestre al usuario un texto que le diga la acción a realizar en dependencia del color que tenga el semáforo (continuar, parar, aminorar)

7. Números Racionales

Dada la implementación de la clase Racional, realice un programa orientado a objetos que:

- Le pida al usuario los valores necesarios para construir dos números racionales R1 y R2.
- Asigne a un nuevo número racional R3 el resultado de sumar R1 con R2 y luego imprima R3 simplificado.
- Asigne al número racional R3 el resultado de restar R1 con R2 y luego imprima R3 simplificado.
- Asigne al número racional R3 el resultado de multiplicar R1 con R2 y luego imprima R3 simplificado.
- Asigne al número racional R3 el resultado de dividir R1 con R2 y luego imprima R3 simplificado.
- Imprima el inverso de R3 simplificado.

Racional	
Funcionalidad	Descripción
- <code>numerador : int</code>	Valor del numerador.
- <code>denominador : int</code>	Valor del denominador.
- <code>Simplificar()</code>	Método que simplifica el número racional, dividiendo el numerador y el denominador por el máximo común divisor (MCD) de ambos.
+ <code>Racional(num: int, den: int)</code>	Construye un número racional a partir de un valor para el numerador y un valor para el denominador.
+ <code>Sumar(N: Racional) : Racional</code>	Realiza la suma entre el número racional que invoca el método y el número racional pasado por parámetro. Retorna el número racional que resulta de dicha operación. El número racional que se retorna ha de estar simplificado.
+ <code>Restar(N: Racional) : Racional</code>	Realiza la diferencia entre el número racional que invoca el método y el número racional pasado por parámetro. Retorna el número racional que resulta de dicha operación. El número racional que se retorna ha de estar simplificado.
+ <code>Multiplicar(N: Racional) : Racional</code>	Realiza la multiplicación entre el número racional que invoca el método y el número racional pasado por parámetro. Retorna el número racional que resulta de dicha operación. El número racional que se retorna ha de estar simplificado.
+ <code>Dividir(N: Racional) : Racional</code>	Realiza la división entre el número racional que invoca el método y el número racional pasado por parámetro. Retorna el número racional que resulta de dicha operación. El número racional que se retorna ha de estar simplificado.
+ <code>Inverso(): Racional</code>	Retorna el inverso del número racional.
+ <code>toString(): string</code>	Retorna una representación en cadena de caracteres del número racional. La representación en cadena ha de tener la forma "a/b" donde <u>a</u> es el numerador y <u>b</u> es denominador del número racional.

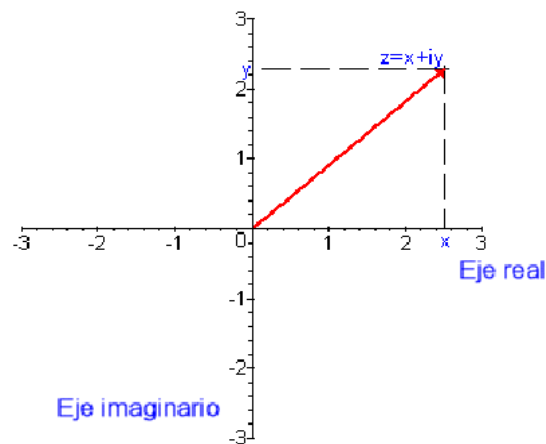
8. Números Complejos

El término **número complejo** describe la suma de un número real y un número imaginario (que es un múltiplo real de la unidad imaginaria, que se indica con la letra **i**).

Los números complejos son una extensión de los números reales, cumpliéndose que los reales son un subconjunto de los complejos. Los números complejos representan todas las raíces de los polinomios, a diferencia de los reales.

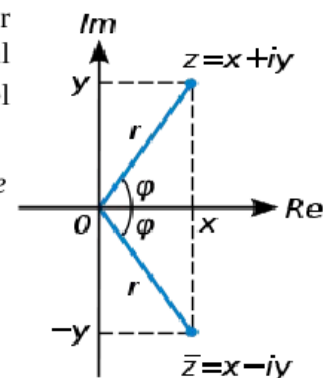
Los números complejos pueden ser representados de varias formas, una de ellas es la binómica, $z = x + yi$, como se muestra en las figuras.

EL PLANO COMPLEJO



Para cada número complejo z , la primera componente, x , es un valor real denominado parte real y la segunda, y , también es un valor real denominado parte imaginaria. La i es la unidad imaginaria del número.

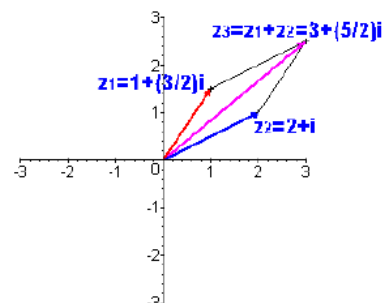
- raíz cuadrada(-1) no está definida en los reales por lo que pasa a ser la unidad imaginaria i .
- -raíz cuadrada(-1) = $-i$
- $i^2 = \text{raíz cuadrada}(-1) = 1$
- $-i^2 = -\text{raíz cuadrada}(-1) = 1$



Al igual que el resto de los números, los complejos tienen operaciones. Las básicas son las siguientes:

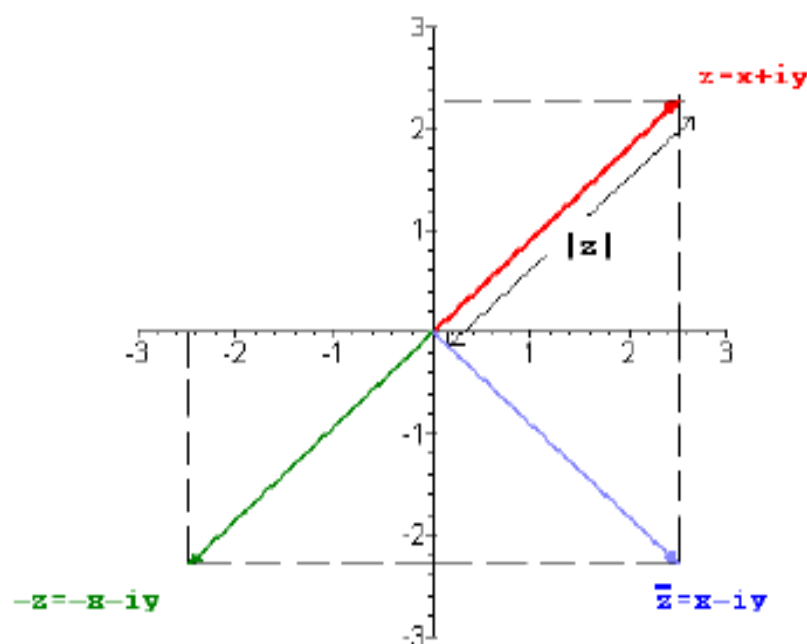
- **Igualdad:** Dos números complejos son iguales si y sólo si lo son simultáneamente sus partes reales y sus partes imaginarias.
Supongamos que tenemos dos números complejos, $Z_1 = X_1 + iY_1$ y $Z_2 = X_2 + iY_2$. $Z_1 = Z_2$ si y solo si $X_1 = X_2$ y $Y_1 = Y_2$.
- **Suma:** La suma de complejos se corresponde con la suma de vectores. Dados dos vectores $Z_1 = X_1 + iY_1$ y $Z_2 = X_2 + iY_2$, su suma es $Z_3 = Z_1 + Z_2 = (X_1 + X_2) + i(Y_1 + Y_2)$. Ver figura a continuación.
- **Resta:** La resta de complejos se corresponde con la resta de vectores. Dados dos vectores $Z_1 = X_1 + iY_1$ y $Z_2 = X_2 + iY_2$, su resta es $Z_3 = Z_1 - Z_2 = (X_1 - X_2) + i(Y_1 - Y_2)$.

SUMA DE COMPLEJOS EN FORMA BINÓMICA



- **Módulo:** Se define el módulo de un número complejo como el módulo del vector que lo representa, es decir, si $Z=X + iY$, entonces el módulo de Z es: $|Z| = \text{Raíz cuadrada}(X^2+Y^2)$.
- **Conjugado:** El conjugado de un número complejo se define como su simétrico respecto del eje real, es decir, si $Z=X + iY$, entonces el conjugado de Z es $Z=X - iY$. Ver figura a continuación.
- **Opuesto:** El opuesto de un número complejo es su simétrico respecto del origen. Ver figura a continuación.
- **Imaginario Puro:** Se denomina **número imaginario puro** a aquel que esta compuesto sólo por la parte imaginaria, es decir, aquel en el que $X = 0$.
- **Neutro:** Se denomina número complejo neutro a aquel complejo que tanto su parte real como la imaginaria son cero, es decir, aquel en el que $X=0$ e $Y=0$.
- **Multiplicación:** Dados dos vectores $Z_1=X_1 + iY_1$ y $Z_2=X_2 + iY_2$, su multiplicación es $Z_3=Z_1*Z_2=(X_1*X_2 - Y_1*Y_2) + i(X_1*Y_2+X_2*Y_1)$.

MÓDULO, CONJUGADO Y OPUESTO DE UN NÚMERO COMPLEJO



Dada la clase NumeroComplejo que se describe a continuación,

NumeroComplejo	
Funcionalidad	Descripción
- parteReal : double	Valor de la parte real del número complejo.
- parteImag : double	Valor de la parte imaginaria del número complejo.
+ NumeroComplejo(parteR:double, parteI: double)	Construye un número complejo a partir de un valor para la parte real y un valor para la parte imaginaria.
+ Sumar(N: NumeroComplejo): NumeroComplejo	Realiza la suma entre el número complejo que invoca el método y el número complejo pasado por parámetro. Retorna el número complejo que resulta de dicha operación.
+ Restar(N: NumeroComplejo): NumeroComplejo	Realiza la diferencia entre el número complejo que invoca el método y el número complejo pasado por parámetro. Retorna el número complejo que resulta de dicha operación.
+ Multiplicar(N: NumeroComplejo): NumeroComplejo	Realiza la multiplicación entre el número complejo que invoca el método y el número complejo pasado por parámetro. Retorna el número complejo que resulta de dicha operación.
+ Dividir(N: NumeroComplejo): NumeroComplejo	Realiza la división entre el número complejo que invoca el método y el número complejo pasado por parámetro. Retorna el número complejo que resulta de dicha operación.
+ ValorAbsoluto(): double	Retorna el valor absoluto del número complejo.
+ Conjugado():NumeroComplejo	Retorna el conjugado del número complejo.
+ toString(): string	Retorna una representación en cadena de caracteres del número complejo. La representación en cadena ha de tener la forma "a + bi" donde a es la parte real y b es la parte imaginaria del número complejo.

Construya un programa orientado a objetos que realice los siguientes pasos:

- Le pida al usuario los valores necesarios para construir dos números complejos C1 y C2.
- Asigne a un nuevo número complejo C3 el resultado de sumar C1 con C2 y luego imprima C3.
- Asigne al número complejo C3 el resultado de restar C1 con C2 y luego imprima C3.
- Asigne al número complejo C3 el resultado de multiplicar C1 con C2 y luego imprima C3.
- Asigne al número complejo C3 el resultado de dividir C1 con C2 y luego imprima C3.
- Imprima el conjugado de C3.
- Imprima el valor absoluto de C3.

9. Barco

Haciendo uso de la clase Barco descrita a continuación,

Barco
<pre> + Barco(maxTon: int, cantTrip: int) + CantTripulantes() : int + MaxToneladas() : int + CantToneladas() : int + Cargar(cantTon : int) : void </pre>

- **CantTripulantes() : int**
Retorna la cantidad de tripulantes que tiene el barco.

- **MaxToneladas() : int**
Retorna la cantidad máxima de toneladas que puede cargar el barco.
- **CantToneladas() : int**
Retorna la cantidad de toneladas que carga actualmente el barco.
- **Cargar(cantTon : int) : void**
Funcionalidad que adiciona una nueva carga al barco. Con el parámetro cantTon se especifica la cantidad de toneladas de carga que se le adicionará al barco.

Desarrolle un programa orientado a objetos que:

- Construya un objeto de la clase Barco con una capacidad máxima de carga y una cantidad de tripulantes entradas por el usuario.
- Cargue el barco con una cantidad de toneladas entradas por el usuario, teniendo en cuenta que en caso de que la cantidad de toneladas entradas por el usuario sea mayor que la cantidad máxima que permite el barco, el programa debe mostrarle al usuario un mensaje de error y no debe efectuarse la carga.
- Determine si con la cantidad de toneladas de carga que posee el barco, se puede hacer el desembarco de la carga en el puerto. Teniendo en cuenta que para mover 10 toneladas hacen falta al menos 15 hombres, para mover 20 son necesarios al menos 35, y para mover más de esa cantidad harían falta más de 50 hombres.

10. Empleado

Dada la clase Empleado desarrolle un programa que construya un empleado a partir de los datos entrados por el usuario y posteriormente:

Calcule el salario del empleado, teniendo en cuenta que para calcular el salario de un empleado se procede de la manera siguiente:

- El salario básico es 245 pesos.
- Si es especialista se le paga además 100 pesos.
- A los líderes se le paga además el 15% de su salario básico.

Dada una cantidad de meses determine si el empleado ha trabajado en la empresa más tiempo que dicha cantidad, tenga en cuenta que todo empleado trabaja al año solo 11 meses (estando el otro de vacaciones).

Empleado	
Funcionalidades	Descripción
+ Empleado(nombre: string, nivel: int, lider: boolean, años: int)	Construye un empleado a partir de un nombre, un nivel (1 o 2), un parámetro booleano que indica si es líder de proyecto o no, y la cantidad de años de experiencia.
+ Nombre(): string	Retorna el nombre del empleado
+ Nivel(): int	Retorna el nivel del empleado.
+ Lider(): boolean	Retorna true si el empleado es líder de proyecto, y false en caso contrario
+ AñosExperiencia(): int	Retorna la cantidad de años de experiencia del empleado.

11. Tiempo

Dada la clase Tiempo que se describe a continuación,

Tiempo

Funcionalidades	Descripción
+Tiempo(tiempo:String)	A partir de una cadena dada en el formato HH:MM:SS construye un objeto tiempo con Hora = HH, Minutos=MM y Segundos=SS.
+ObtenerHora():int	Retorna el valor correspondiente a la Hora.
+ObtenerMinutos():int	Retorna el valor correspondiente a los Minutos.
+ObtenerSegundos():int	Retorna el valor correspondiente a los Segundos.

desarrolle un programa que:

- Muestre el saludo correspondiente a un horario entrado por el usuario en el formato Hora:Minutos:Segundos.
- Agregue al programa la posibilidad de que el usuario entre el horario que desea consultar hasta que introduzca la letra 'S'.

12. Catálogo histórico

Dada la clase CatálogoHistórico que se describe a continuación,

CatalogoHistorico	
Funcionalidades	Descripción
<code>CatalogoHistorico(cantMaxHechosHistoricos: int)</code>	Construye un catálogo histórico vacío, con una capacidad máxima para almacenar hechos históricos igual a <code>cantMaxHechosHistoricos</code> .
<code>adicionarHechoHistorico(FechaHistorica:String,acontecimiento:String): void</code>	Adiciona un nuevo hecho histórico al catálogo. Siempre que la cantidad de hechos históricos adicionados no exceda el valor de <code>cantMaxHechosHistoricos</code> .
<code>ObtenerAcontecimiento(Fecha:String):String</code>	Retorna el acontecimiento asociado a la fecha introducida por el usuario. Si la fecha no es histórica entonces se retorna la cadena vacía ("").
<code>ObtenerAcontecimientos(fechaIncial:String, fechaFinal:String):String</code>	Retorna, en una cadena, las fechas y los respectivos acontecimientos que se encuentran dentro del rango de fecha pasado por parámetro, siguiendo el siguiente formato: <code><fechaIncial> -<fechaFinal></code> <code><Fecha Histórica>:<Acontecimientos></code> Retorna la cadena vacía ("") si no existen fechas históricas entre el rango de fechas pasadas como parámetros.

Desarrolle un programa que:

- Cree un catálogo Histórico con una capacidad máxima de hechos históricos entrada por el usuario.
- Adicione, al catalogo histórico, una cantidad de hechos históricos introducidos por el usuario, siempre que sea posible.
- Muestre el acontecimiento correspondiente a una fecha histórica, con formato DD/MM/AAAA, entrada por el usuario. Si la fecha entrada no está registrada como histórica en el catálogo histórico entonces se debe mostrar un mensaje que informe esta situación.
- Muestre todos los acontecimientos históricos con sus respectivas fechas que se encuentran en un rango de fecha (fecha inicial y fecha final) entrado por el usuario.
- Agregue al programa la posibilidad de que el usuario entre rangos de fechas que desea consultar hasta que introduzca la letra 'S'.

13. Automóvil 2

Dada la clase Automóvil que se describe a continuación,

Automóvil	
Funcionalidad	Descripción
<code>Automóvil()</code>	Construye un automóvil.
<code>Arrancar()</code>	Arranca el automóvil. La funcionalidad <code>Acelerar</code> no se puede usar sin antes haber arrancado el automóvil.
<code>Acelerar()</code>	Acelera el automóvil en razón de 1 Km/h. Es decir, incrementa la velocidad que actualmente tiene el automóvil en 1 Km/h
<code>VelocidadMax(): int</code>	Retorna la velocidad máxima que puede alcanzar el automóvil.
<code>VelocidadActual(): int</code>	Retorna la velocidad que ha alcanzado hasta el momento el automóvil.
<code>KmsPorLitro(): int</code>	Retorna la cantidad de kilómetros que puede recorrer el automóvil por cada litro de combustible.

Desarrolle un programa que permita:

- Calcular la cantidad de combustible necesaria para recorrer una cantidad de kilómetros entrada por el usuario.
- Acelerar el automóvil hasta que llegue a una velocidad límite entrada por el usuario, o hasta que llegue a su velocidad máxima.

14. Impresora

Dada la clase Impresora que se describe a continuación,

Impresora	
Atributos y métodos	Descripción
- <code>cantHojas : int</code>	Cantidad de hojas que tiene en la bandeja la impresora.
- <code>encendida : boolean</code>	Es <code>true</code> si la impresora está encendida, en caso contrario es <code>false</code> .
- <code>vacía : boolean</code>	Indica si la impresora está vacía.
+ <code>Impresora()</code>	Construye una impresora con una capacidad en la bandeja para 100 hojas.
+ <code>Encender()</code>	Enciende la impresora. La funcionalidad <code>Imprimir</code> no se puede usar sin antes haber encendido la impresora.
+ <code>Apagar()</code>	Apaga la impresora.
+ <code>Imprimir()</code>	Imprime una hoja. Disminuye en 1 la cantidad de hojas que tiene la impresora en su bandeja. Tiene que tener en cuenta que la impresora no esté vacía de hojas.
+ <code>PonerHojas(cantHojas:int)</code>	Pone una cantidad de hojas en la bandeja de la impresora. Debe tener en cuenta que ya pueden haber hojas en la bandeja.
+ <code>EstaVacía(): boolean</code>	Retorna <code>true</code> si ya no quedan hojas en la bandeja, y <code>false</code> en caso contrario.
+ <code>CantidadHojas(): int</code>	Retorna la cantidad de hojas que tiene en la bandeja.

Desarrolle un programa que simule el proceso de impresión de un documento de una cantidad de hojas determinadas por el usuario. Siempre que se vacíe la bandeja de la impresora, y aún el documento no se ha terminado de imprimir, el programa debe pedirle al usuario la cantidad de hojas que desea colocar en la bandeja, y luego proseguir con la impresión.

15. Automóvil 3

Dada la clase Automóvil que se describe a continuación,

Automovil	
Características y funcionalidades	Descripciones
- velocidadMax: int	almacena la velocidad máxima a la que puede avanzar el auto.
- kmPorLitros: int	almacena la cantidad de kilómetros que puede recorrer el auto con un litro de combustible.
- capacidadTanque: int	almacena la cantidad máxima de combustible que se puede almacenar en el tanque del auto.
- velocidadActual: int	Almacena la velocidad a la que está avanzando el auto.
- cantidadCombustible: int	Almacena la cantidad de combustible que tiene el auto almacenada en el tanque.
- encendido: boolean	almacena true si el auto está encendido y false en caso contrario.
+ Autimovil(velocidadMax: int, kmPorLitros: int, capacidadTanque: int)	Construye un automóvil con una velocidad máxima, kilómetros por litros y capacidad del tanque pasada por parámetros. Inicialmente el auto está apagado y el tanque de combustible está vacío.
+ VelocidadActual(): int	Retorna la velocidad a la que está avanzando el auto.
+ CantidadComb(): int	Retorna la cantidad de combustible que tiene el auto almacenada en el tanque.
+ Encendido():boolean	Retorna true si el auto está encendido y false en caso contrario.
+ KmporLitros(): int	Retorna la cantidad de kilómetros que puede recorrer el auto con un litro de combustible.
+ Encender(): void	Modifica a true el estado de la variable encendido.
+ Acelerar(): void	Incrementa a razón de 1 la velocidad del auto siempre que sea posible. Un automóvil puede ser acelerado si su velocidad aún no ha alcanzado el límite de velocidad.
+ Apagar(): void	Modifica a false el estado de la variable encendido.
+ AgregarCombustible(cantCombustible: int): boolean	Aumenta la cantidad de combustible del auto en una cantidad pasada por parámetro, si cabe en el tanque del automóvil. Retorna si se pudo o no agregar combustible al tanque.
+ Avanzar():void	Decrementa, del tanque, la cantidad de combustible necesaria para recorrer 1km si esto es posible. Si no es posible el automóvil debe detenerse.

Desarrolle un programa principal que realice las siguientes actividades:

- Construir un objeto nombrado *auto1* de la clase *Automóvil* con una capacidad máxima del tanque, velocidad máxima del auto y una cantidad de kilómetros por litros entradas por el usuario.
- Acelerar y echar a andar el auto hasta que se quede sin combustible o recorra una cantidad de kilómetros entrada por el usuario. Tenga en cuenta que para acelerar o poner a andar a un auto este tiene que estar encendido y tener combustible en el

tanque.

- Mejore el inciso anterior para que el auto esté andando hasta que recorra la cantidad de kilómetros entrado anteriormente por el usuario.
- Cuando el auto llegue a la meta debe apagarse si aún está encendido.
- Modifique el programa anterior para que muestre en consola la cantidad de veces que se le tuvo que adicionar combustible al auto para que lograra llegar a su destino.

16. Máquina expendedora de Café

Se quiere modelar una máquina expendedora de café que ofrezca los siguientes productos:

- Café fuerte (Tipo 1): Consume en su elaboración 0,5 gramos de café molido.
- Café claro (Tipo 2): Consume en su elaboración 0,3 gramos de café molido.
- Café con leche (Tipo 3): Consume en su elaboración 0,1 gramo de café molido y para ello se ha diseñado la siguiente clase.

Haciendo uso de la clase MáquinaCafé,

MáquinaCafé	
Funcionalidad	Descripción
+MáquinaCafé(cantMaxGramos:double)	Construye una máquina de café con una capacidad máxima para almacenar el café molido (dado en gramos) que puede almacenar en su interior. Inicialmente la máquina no tiene café molido almacenado en su interior.
+Expende(tipoCafé:int):void	Entrega un vaso con un tipo de café pasado por parámetro. Recibe como parámetro el tipo de café a entregar (que tendría valor 1, 2 ó 3). La cantidad de gramos de café molido que se requiere en su elaboración es sustraída de la cantidad de gramos de café molido existentes en el interior de la máquina.
+Reponer():double	Llena la máquina de café molido a toda su capacidad y retorna la cantidad de dinero recaudado.
+PuedeExpende(tipoCafé:int):boolean	Recibe como parámetro un tipo de café (que tendría valor 1, 2 ó 3). Retorna true si la máquina puede expender un café de este tipo, es decir, si la cantidad de gramos de café molido existente en el interior de la máquina es suficiente para elaborar el tipo de café solicitado. Retorna false en caso contrario.
+CantMaxGramos():int	Retorna la capacidad máxima de gramos de café molido que admite la máquina.

desarrolle un programa que:

- Construya un objeto de la clase MáquinaCafé. La capacidad máxima de gramos de café molido que puede almacenar en su interior debe ser entrada por el usuario.
- Le pregunte al usuario el tipo de café que desea tomar y realice la venta de un vaso de café del tipo deseado. Cuando se entra un tipo de café, se debe expender el café siempre. En caso que no sea posible porque la cantidad de gramos en el interior de la máquina no es suficiente para su elaboración, entonces se debe, antes de expender el café, llenar nuevamente el recipiente de la máquina y mostrar al usuario cuánto se ha recaudado.

- Agregar al programa la posibilidad de que la máquina realice las acciones antes descritas hasta que el usuario oprima el botón de salir (introduzca la letra 'S').

17. Transportador de cargas pesadas

Se ha diseñado una clase para modelar un transportador de cargas pesadas. El transportador puede trasladar al mismo tiempo varios objetos de distintos pesos, siempre y cuando el peso total cargado sea menor que un peso máximo de carga. El diseño y la especificación de la clase es el siguiente:

Transportador	
Funcionalidades	Descripción
+Transportador(pesoMaximo: double)	Construye un transportador con un peso máximo que puede admitir para todo traslado. Inicialmente el transportador no tiene carga alguna.
+ Cargar(pesoCarga: double)	Coloca una carga en el transportador. Recibe como parámetro el peso de dicha carga, el cual debe ser adicionado al peso acumulado del transportador.
+ EntregarCarga()	El transportador traslada y entrega todos los objetos que carga en ese momento. El peso acumulado del transportador se hace cero.
+ PuedeCargar(pesoCarga: double): boolean	Recibe como parámetro el peso de una carga. Retorna true si puede colocarse en el transportador la carga correspondiente al peso pasado por parámetro, es decir, si la carga acumulada junto con la carga pasada por parámetro no excede el peso máximo admitido. Retorna false en caso contrario.
+ PesoMaximo(): int	Retorna el peso máximo que puede admitir el transportador.

Desarrolle un programa que:

- Construya un objeto de la clase Transportador con un peso máximo de carga entrado por el usuario.
- Realice la carga de un nuevo objeto cuyo peso debe ser pedido al usuario, teniendo en cuenta que cuando se entre el peso del objeto, se debe colocar el objeto en el transportador siempre y cuando sea posible colocarlo. En caso que no sea posible colocar el objeto porque puede excederse el peso máximo admitido, el transportador debe proceder a entregar la carga que tiene, y posteriormente se debe colocar el objeto en el transportador.
- Haga que el proceso antes descrito se realice para una cantidad de objetos entrados por el usuario.

Nota: Considere que los pesos, entrados por el usuario, de cada objeto a cargar será menor o igual al peso máximo admitido por el transportador.

18. Control de nómina

El departamento de personal de la UCI desea contar con un programa para llevar el control de la nómina de cada trabajador. En él se ha decidido representar a los Trabajadores de Servicio mediante su nombre, salario diario, la cantidad de días trabajados y la evaluación del desempeño (superior (1), adecuado (2), deficiente (3)).

El salario se calcula de la siguiente manera:

- Si es deficiente: $\text{salario diario} * \text{días trabajados} - (\text{salario diario} / 4) * \text{días trabajados}$
 - Si es adecuado: $\text{salario diario} * \text{días trabajados}$
 - Si es superior: $\text{salario diario} * \text{días trabajados} + (\text{salario diario} / 3) * \text{días trabajados}$
- Haciendo uso de la clase,

TrabajadorDeServicio	
Funcionalidades	Descripción
+ <code>TrabajadorDeServicio(nombre: string, salDiario: double, cantDias: int, eval: int)</code>	Construye un trabajador de servicio a partir de un nombre, un salario diario, una cantidad de días trabajados, y su evaluación (1, 2, 3)
+ <code>Nombre(): string</code>	Retorna el nombre del trabajador
+ <code>SalarioDiario(): int</code>	Retorna el salario diario del trabajador.
+ <code>CantDias(): int</code>	Retorna la cantidad de días trabajados
+ <code>Evaluacion(): int</code>	Retorna la evaluación del trabajador.

desarrolle un programa que construya un trabajador de servicio a partir de datos entrados por el usuario, y posteriormente:

- Calcule el salario de un trabajador de servicio.
- Dado un par de números que representan dos cantidades de horas determine si el empleado ha trabajado un tiempo que esté en ese rango.
- Haga que el proceso antes descrito se realice para una cantidad de trabajadores entrados por el usuario.

19. Diccionario

Dada la clase diccionario que se describe a continuación,

+ Diccionario
+ Diccionario(cantMaxPalabras:int) + adicionar_palabra(palabra:String):void + cantPalabras(letra:char):int + palabra_significado(letra: char, posicion: int):String + primeraPalabra(palabra_significado:String) :String + terminaEn(palabra:String, terminacion:String):boolean

- **Diccionario(cantMaxPalabras:int)**
Permite construir un diccionario que podrá tener como máximo una cantidad de palabras, *cantMaxPalabras*.
- **Adicionar_Palabra(palabra:String):void**
Permite adicionar una *palabra* al diccionario, esta palabra debe cumplir el siguiente formato: *palabra:significado*.
- **CantPalabras(letra:char):int**
Devuelve cuántas palabras hay en el diccionario que empiezan con la letra *letra*.
- **Palabra_Significado(letra: char, posicion: int):String**
Devuelve de entre todas las palabras que empiezan con la letra *letra*, la número *posición*.
Ejemplo: si el diccionario tiene 20 palabras que empiezan con la letra 'A' y yo pido

palabra_significado('A', 5) me devuelve la 5ta palabra que comienza con 'A' de entre las 20, en el formato **palabra:significado**.

- **PrimeraPalabra(palabra_significado:String) :String**

Dada la palabra **palabra** en el formato **palabra:significado**, devuelve **palabra**. Es decir, sin el significado y los dos puntos(:).

- **TerminaEn(palabra:String, terminacion:String):boolean**

Dada la palabra **palabra**, sin el significado ni los dos puntos, y una terminación **terminacion** devuelve verdadero si **palabra** termina en **terminacion** sino devuelve falso.

Realice un programa que:

- Cree un diccionario con una capacidad de palabras entrada por el usuario.
- Adicione al diccionario tantas palabras(en el formato palabra:significado) como especifique el usuario que va a entrar. Tenga en cuenta que la cantidad de palabras que quiera entrar el usuario tiene que estar acotada superiormente por la capacidad máxima de palabras que puede tener el diccionario e inferiormente por 0.
- Adicione a un nuevo diccionario solo aquellas palabras que empiezan y terminan con una letra y una terminación entrada por el usuario, respectivamente.
- Muestre al usuario, la cantidad de palabras con que cuenta el nuevo diccionario.

20. Máquina de Pinturas

Dada la clase MáquinaDePinturas que se describe a continuación,

MáquinaPinturas	
Atributos y métodos	Descripción
- <u>capacidadMaxL : double</u>	Cantidad máxima de litros que puede almacenar cada depósito de la máquina.
- <u>cantLitrosRojo : double</u>	Cantidad de litros de pintura que tiene el depósito para el color rojo.
- <u>cantLitrosAmarillo : double</u>	Cantidad de litros de pintura que tiene el depósito para el color amarillo.
- <u>cantLitrosAzul : double</u>	Cantidad de litros de pintura que tiene el depósito para el color azul.
+ <u>MáquinaPinturas (maxCapacidadL: double)</u>	Construye una máquina dispensadora de pinturas con una cantidad máxima de litros para cada depósito. Inicialmente ningún depósito tiene pinturas en su interior.
+ <u>Reponer()</u>	Llena cada depósito de la máquina a toda su capacidad.
+ <u>Dispensar(pRojo, pAmarillo, pAzul: double)</u>	Dispensa un litro de pintura, que se obtiene de disolver los colores rojo, amarillo y azul según los respectivos <u>porcientos</u> pasados por parámetro (<u>pRojo</u> , <u>pAmarillo</u> y <u>pAzul</u>). Disminuye la cantidad de litros de cada pintura, según la cantidad utilizada en la disolución.
+ <u>PuedeDispensar(pRojo, pAmarillo, pAzul: double): boolean</u>	Retorna <u>true</u> si se puede hacer 1 litro de pintura con los <u>porcientos</u> de rojo, amarillo y azul pasados por parámetro, y <u>false</u> en caso contrario. El litro de pintura no se puede hacer si la cantidad existente de algún color es menor que la requerida por el usuario.
+ <u>ColorLimitado(): String</u>	Devuelve una cadena de caracteres que indica el color ("Rojo", "Amarillo", "Azul") cuya cantidad en la máquina sea la menor.
+ <u>CapacidadMaxL(): int</u>	Retorna el valor del atributo <u>capacidadMaxL</u> .

Desarrolle un programa con los siguientes requerimientos:

- Construya un objeto de la clase MáquinaPinturas con una cantidad máxima de litros entrada por el usuario.

- Dispense al usuario litros de pintura hasta que éste no desee ninguna más. Para ello el programa, luego que la máquina haya dispensado un litro de pintura, debe preguntarle al usuario si desea otro más. Si el usuario responde que sí ('S') entonces se repite el proceso de dispensar y volver a preguntar, y si responde que no ('N') entonces no se dispensará más ninguna.
- Antes de dispensar algún litro de pintura, el programa le debe pedir al usuario los porcentajes de rojo, amarillo y azul que desea para la disolución.
- Cuando no es posible dispensar un determinado pedido del usuario, el programa debe llenar los depósitos de la máquina hasta su máxima capacidad.

21. Máquina de Helados

Dada la clase MáquinaHelado descrita a continuación,

MáquinaHelados	
Atributos y métodos	Descripción
- <u>cantMaxBolas</u> : <u>int</u>	Cantidad máxima de bolas de helado que puede almacenar cada tina de la máquina.
- <u>cantBolasCH</u> : <u>int</u>	Cantidad de bolas que tiene la tina de sabor chocolate.
- <u>cantBolasF</u> : <u>int</u>	Cantidad de bolas que tiene la tina de sabor fresa.
- <u>cantBolasV</u> : <u>int</u>	Cantidad de bolas que tiene la tina de sabor vainilla.
+ <u>MáquinaHelados(cantMaxBolas: int)</u>	Construye una máquina dispensadora de helados con una cantidad máxima de bolas para cada tina. Inicialmente ninguna tina tiene helados en su interior.
+ <u>Reponer()</u>	Llena cada tina de la máquina a toda su capacidad.
+ <u>Dispensar(bolasCH: int, bolasF: int, bolasV: int)</u>	Dispensa un plato de helados conformado con las cantidades pasadas por parámetros. La dispensación consiste en disminuir la cantidad de bolas de cada sabor, en las cantidades pasada por parámetro.
+ <u>PuedeDispensar(bolasCH: int, bolasF: int, bolasV: int): boolean</u>	Retorna <u>true</u> si se puede dispensar el plato conformado por las cantidades de bolas de cada sabor especificadas por parámetro. Retorna <u>false</u> en caso contrario. El plato pedido no se puede confeccionar si la cantidad existente de bolas de algún sabor es menor que la requerida por el plato.
+ <u>SaborLimitado(): String</u>	Devuelve una cadena de caracteres que indica el sabor ("Chocolate", "Fresa", "Vainilla") cuya cantidad en la máquina sea la menor.
+ <u>CantMaxBolas(): int</u>	Retorna el valor del atributo <u>cantMaxBolas</u> .

Desarrolle un programa con los siguientes requerimientos:

- Construya un objeto de la clase MáquinaHelados con una cantidad máxima de bolas entrada por el usuario.
- Dispense al usuario platos de helado hasta que éste no desee ninguna más. Para ello el programa, luego que la máquina haya dispensado un plato, debe preguntarle al usuario si desea otro más. Si el usuario responde que sí ('S') entonces se repite el proceso de dispensar y volver a preguntar, y si responde que no ('N') entonces no se dispensará más ninguna.
- Antes de dispensar algún plato de helados, el programa le debe pedir al usuario la cantidad de bolas que desea por cada sabor.
- Cuando no es posible dispensar un determinado pedido del usuario, el programa debe llenar las tinas de la máquina hasta su máxima capacidad.

22. Máquina de Peluches

Dada la clase MaquinaDePeluches descrita a continuación,

<u>MaquinaDePeluches</u>	
Atributos y métodos	Descripción
- <u>CantMaxPeluches</u> : <u>int</u>	Cantidad máxima de peluches que puede almacenar la bandeja de la máquina.
- <u>CantPeluches</u> : <u>int</u>	Cantidad de peluches que tiene la bandeja de la máquina en un momento determinado.
- <u>intento</u> : <u>int</u>	Cantidad de veces que se ha intentado agarrar un peluche.
+ <u>MaquinaDePeluches</u> (<u>CantMaxPeluches</u> : <u>int</u>)	Construye una máquina con una cantidad máxima de peluches que puede almacenar su bandeja. Inicialmente la máquina no tiene peluches y no se ha intentado obtener peluches.
+ <u>SostenerPeluche</u> (): <u>boolean</u>	Retorna <u>true</u> si el brazo mecánico pudo sostener el peluche y <u>false</u> en caso contrario. Se pudo sostener el peluche si la fuerza del brazo es mayor que el peso del peluche seleccionado. Se logre o no sostener el peluche se cuenta como un intento. La fuerza del brazo se calcula como: $((\text{intento} \% 11) / 10) * \text{Peso de un Peluche} + 1$.
+ <u>SoltarPeluche</u> (): <u>void</u>	Entrega el peluche al usuario.
+ <u>ColocarPeluches</u> (<u>cantPeluches</u> : <u>int</u>): <u>void</u>	Coloca, en la bandeja de la máquina, una cantidad de peluches de 50 gramos cada uno.
+ <u>MoverBrazo</u> (<u>idPeluche</u> : <u>int</u>): <u>void</u>	Mueve el brazo en dirección al peluche con identificador igual al entrado.
+ <u>CantMaxPeluches</u> (): <u>int</u>	Retorna la cantidad máxima de peluches que admite la bandeja.
+ <u>Vacia</u> (): <u>boolean</u>	Retorna <u>true</u> si la bandeja no tiene peluches y <u>false</u> en caso contrario.

Desarrolle un programa que:

- Construya un objeto de la clase *MaquinaDePeluches* con una cantidad máxima de peluches entradas por el usuario.
- Simule el juego de obtener peluches, de uno en uno, hasta que el usuario entre la opción de no seguir jugando (entrar el caracter N).
- Antes de seleccionar cada peluche se pida al usuario el identificador del peluche que desea obtener.
- Cuando la bandeja se haya vaciado coloque en ella el máximo números de peluches y posteriormente siga jugando.
- Muestre al usuario cuántos peluches pudo llevarse a la casa.

23. Lanza Pelotas

Se ha diseñado la siguiente clase para modelar una máquina que lanza pelotas de béisbol. Estas máquinas son usadas habitualmente en los entrenamientos de bateo para sustituir la figura del lanzador. Ellas tienen una bandeja de pelotas, de donde escogen una pelota para realizar un lanzamiento. Dada la siguiente descripción declare e implemente la clase *LanzaPelotas*.

LanzaPelotas	
Atributos y métodos	Descripción
-cantMaxPelotas : int	Cantidad máxima de pelotas que puede almacenar la bandeja de la máquina.
-cantPelotas : int	Cantidad de pelotas que tiene la bandeja de la máquina en un momento determinado.
+ Lanzapelotas(cantMaxPelota : int)	Construye una maquina con una cantidad máxima de pelotas que debe almacenar su bandeja.
+ PonerPelotas(cantPelotas: int)	Pone una cantidad de pelotas en la bandeja de la maquina (cantPelotas). Debe tener en cuenta que ya puede haber pelotas en la bandeja, y que al final el total de pelotas en la bandeja no puede exceder la cantidad máxima que permite.
+ Lanzar()	Lanza una pelota. Disminuye en 1 la cantidad de pelotas en la bandeja de la máquina.
+ BandejaVacía(): boolean	Retorna true si ya no quedan pelotas en la bandeja de la maquina y false en caso contrario.
+ CantMaxPelotas(): int	Retorna la cantidad máxima de pelotas que permite la bandeja de la máquina.

Desarrolle un programa un programa que:

- Construya un objeto de la clase LanzaPelotas con una cantidad máxima de pelotas entradas por el usuario.
- Le pida al usuario la cantidad de lanzamientos que desea realice la maquina lanza pelotas.
- Realice la cantidad de lanzamientos entrada por el usuario. Cuando se a necesario poner pelotas en la bandeja porque esta se ha vaciado, debe colocar en la bandeja el máximo números de pelotas y posteriormente lanzar una pelota.

24. Fusil

La gran mayoría de los fusiles modernos tienen un dispositivo llamado comúnmente “cargador”, en el cual se colocan las balas que puede disparar el fusil. Siempre que se agoten las balas del cargador del fusil, y se quiera seguir disparando, es necesario entonces volver a colocar balas en el cargador. Dada la siguiente descripción declare e implemente la clase Fusil.

Fusil	
Atributos y métodos	Descripción
-cantMaxBalas : int	Cantidad máxima de balas que permite el cargador del fusil.
-cantBalas : int	Cantidad de balas que tiene el cargador del fusil en un momento determinado.
+ Fusil(cantMaxBalas : int)	Construye un fusil con una cantidad máxima de balas que debe permitir su cargador.
+ Cargar(cantBalas: int)	Coloca un cargador al fusil con una determinada cantidad de balas (cantBalas). Dicha cantidad de balas no puede ser mayor que la cantidad máxima de balas que permite el cargador
+ Disparar()	Realiza un disparo. Disminuye en 1 la cantidad de balas en el cargador.
+ CargadorVacio(): boolean	Retorna true si ya no quedan balas en el cargador del fusil y false en caso contrario.
+ CantMaxBalas(): int	Retorna la cantidad máxima de balas que permite el cargador de fusil.

Desarrolle un programa un programa que:

- Construya un objeto de la clase Fusil con una cantidad máxima de balas entradas por el usuario.
- Le pida al usuario la cantidad de disparos que desea realizar con el fusil.
- Realice la cantidad de disparos entrada por el usuario. Cuando se a necesario cargar el fusil porque se le han agotado las balas, debe realizar la carga con el máximo números de balas que permite el cargador.

25. Dados

Un dado es un objeto de forma cúbica, que en cada una de sus caras tiene puntos negros que en total será un número del 1 al 6. Por lo general, lo que más interesa a la hora de usar un dado es el valor que presenta la cara superior luego de una tirada.



Se quiere realizar la programación de un juego que juegan dos jugadores, y que consiste en que en cada turno cada jugador tira un dado. El turno lo gana el jugador que más puntos alcanzó con su dado, y queda empatado el turno cuando ambos obtuvieron el mismo número con su dado. Finalmente el juego lo gana aquel jugador que más turnos ganó.

Para este sistema, se ha diseñado la siguiente clase:

Jugador	
Funcionalidad	Descripción
<code>Jugador(nombre: string)</code>	Construye un jugador con un determinado nombre.
<code>TirarDado(): int</code>	Retorna un valor entero que representa la cantidad de puntos que tiene la cara superior del dado luego de ser tirado por el jugador.
<code>Nombre(): string</code>	Retorna el nombre del jugador.

Desarrolle un programa que simule este juego, y por tanto:

- Le pida al usuario los nombres de los dos jugadores, así como la cantidad de turnos que desea que ambos jugadores jueguen. Los dos tiradores deben ser contruidos con los nombres entrados por el usuario.
- Haga jugar a ambos jugadores durante la cantidad de turnos entrada por el usuario. En cada turno debe mostrarse en pantalla el resultado de la tirada de cada jugador.
- Luego de agotarse todos los turnos, mostrar el nombre del jugador que más turnos ganó.

26. Piedra-Papel-Tijera

El conocido juego Piedra-Papel-Tijera consiste en que dos jugadores, al mismo tiempo, hacen una de estas tres señales con una de sus manos:



Piedra



Papel



Tijera

Un jugador le gana el turno al otro si:

- Su mano señalizó una piedra y la de su contrario señalizó una tijera.
- Su mano señalizó un papel y la de su contrario señalizó una piedra.
- Su mano señalizó una tijera y la de su contrario señalizó un papel.

Para el desarrollo de un sistema que simule este juego, se ha diseñado la siguiente clase:

Jugador	
Funcionalidad	Descripción
Jugador(nombre: string)	Construye un jugador con un determinado nombre.
Señalizar(): int	Retorna un valor entero en dependencia de la señal realizada por el jugador. Retorna 1 si es piedra, 2 si es papel y 3 si es tijera.
Nombre(): string	Retorna el nombre del jugador.

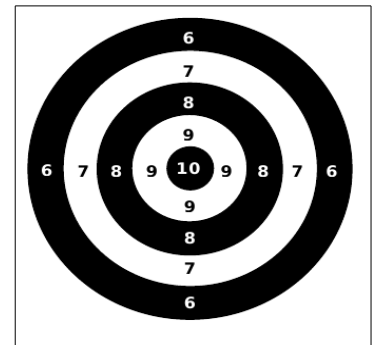
Desarrolle un programa que simule este juego, y por tanto:

- Le pida al usuario los nombres de los dos jugadores, así como la cantidad de turnos que desea que ambos jugadores jueguen. Los dos tiradores deben ser contruidos con los nombres entrados por el usuario.
- Haga jugar a ambos jugadores durante la cantidad de turnos entrada por el usuario. En cada turno debe mostrarse en pantalla lo que cada jugador escogió señalar.
- Luego de agotarse todos los turnos, mostrar el nombre del jugador que más turnos ganó.

27. Tiro con Arco

Una competencia de tiro con arco se compone de varias rondas, y en cada ronda cada competidor realiza un disparo a una diana que tiene la siguiente forma:

En dependencia de donde caiga la flecha, el tirador recibe una puntuación entre 6 y 10. En caso que la flecha disparada no impacte la diana, el tirador no recibe puntos.



Para desarrollar un sistema que simule una competencia de tiro con arco, se diseñó la siguiente clase:

Tirador	
Funcionalidad	Descripción
Tirador(nombre: string)	Construye un tirador con un determinado nombre.
Disparar(): int	Retorna un valor entero en dependencia del lugar de la diana donde dio el disparo, por lo que el valor puede ser 6, 7, 8, 9 y 10. Retorna 0 en caso que el disparo no impacte la diana.
Nombre(): string	Retorna el nombre del jugador.

Desarrolle un programa que simule una competencia de tiro al arco entre dos tiradores, y por tanto:

- Le pida al usuario los nombres de los dos tiradores, así como la cantidad de rondas que

compondrá la competencia. Los dos tiradores deben ser construidos con los nombres entrados por el usuario.

- Haga disparar a ambos tiradores durante la cantidad de rondas entrada por el usuario. En cada ronda debe mostrarse en pantalla la puntuación obtenida por el tirador.
- Luego de agotarse todas las rondas, mostrar el nombre del tirador que más puntos acumuló.

28. Computadora

Haciendo uso de la clase Computadora que se describe a continuación:

Computadora	
Características y Funcionalidades	Descripción
- <u>capacidadMaxDisco</u> : <u>double</u>	Cantidad máxima de <u>Gigabytes</u> que puede almacenar el disco duro de la computadora.
- <u>capacidadOcupadaDisco</u> : <u>double</u>	Cantidad de <u>Gigabytes</u> que tiene ocupada el disco duro de la computadora.
+ <u>Computadora</u> (<u>capacidadMaxDisco</u> : <u>double</u>)	Construye una computadora con capacidad máxima de disco duro igual a una cantidad de <u>Gigabytes</u> pasada por parámetro y con 5 <u>Gigabytes</u> ocupados en cuestiones del sistema operativo que tiene instalado.
+ <u>EstaDisponible</u> () : <u>boolean</u>	Retorna <u>true</u> si la computadora está disponible. Retorna <u>false</u> en otro caso.
+ <u>ExisteFichero</u> (<u>nombreFichero</u> : <u>String</u>) : <u>boolean</u>	Retorna <u>true</u> si el fichero con nombre igual al pasado por parámetro existe en la computadora. Retorna <u>false</u> en otro caso.
+ <u>ObtenerTamanoFicheroGB</u> (<u>nombreFichero</u> : <u>String</u>) : <u>double</u>	Retorna el tamaño en <u>Gigabytes</u> del fichero con nombre igual al pasado por parámetro. Si el fichero no existe en la computadora, esta acción no debe ser ejecutada.
+ <u>PuedeCopiar</u> (<u>tamanoFicheroGB</u> : <u>double</u>) : <u>boolean</u>	Retorna <u>true</u> si la computadora tiene espacio suficiente sin ocupar para almacenar una cantidad de <u>Gigabytes</u> igual al tamaño del fichero pasado por parámetro. Retorna <u>false</u> en otro caso.
+ <u>Copiar</u> (<u>nombreFichero</u> : <u>String</u> , <u>tamañoFicheroGB</u> : <u>double</u>) : <u>void</u>	Método que copia, en el disco duro de la computadora, un fichero con nombre y tamaño en <u>Gigabytes</u> pasados por parámetro.

Desarrolle un programa que:

- Construya dos computadoras, una PC_Origen y una PC_Destino, dada la capacidad en disco(en Gigabytes) de cada una.
- Realice el proceso de copiar, desde la PC_Origen a la PC_Destino, cada uno de los ficheros con nombres entrados por el usuario. Dada la cantidad de ficheros que se quieren copiar.
- Muestre la cantidad de ficheros que pudieron ser copiados y la cantidad total de Gigabytes que ocupan estos ficheros en la computadora.

29. Estantes

Haciendo uso de la clase Estante que se describe a continuación:

Estante	
Características y Funcionalidades	Descripción
- <u>capacidadEstante: int</u>	Cantidad de libros que caben en el estante en cada instante de tiempo.
- <u>libros: String</u>	Atributo en el que se almacena, entre llaves y separada por coma, la información de los libros almacenados en el estante. De cada libro se gestiona, como una cadena, el título del libro y su categoría en el formato siguiente: <Título del libro> : <Categoría> <u>Ejemplos:</u> 1. "{Introducción a la POO : Especialidad, Programación en C++ : Especialidad, Física : Cálculo}" 2. "{}"
+ <u>Estante(capacidadEstante: int)</u>	Construye un estante vacío con una capacidad para libros igual al valor pasado por parámetro.
+ <u>Estante(capacidadEstante: int, cantLibros: int)</u>	Construye un estante con una capacidad para libros igual a la capacidad del estante pasada por parámetro menos la cantidad de libros que existen en el estante. Si <u>cantLibros</u> es cero entonces se construye un estante vacío, sin libros
+ <u>AñadirLibro(libro: String): void</u>	Coloca el libro pasado por parámetros detrás del último libro que está almacenado en el estante.
+ <u>Lleno(): boolean</u>	Retorna <u>true</u> si el estante no tiene capacidad para almacenar libros y <u>false</u> en caso contrario.
+ <u>ObtenerCategoríaDeLibro(): int</u>	<u>Retorna un valor entero en correspondencia con la categoría del primer libro que está almacenado en el estante. Retorna 1 si es Especialidad, 2 si es Cálculo y 3 si es Letras.</u>
+ <u>Vacio(): boolean</u>	Retorna <u>true</u> si el estante no contiene libros y <u>false</u> en caso contrario.
+ <u>Extraer(): String</u>	Extrae el primer libro que está almacenado en el estante. Es decir, elimina el libro que más tiempo lleva en el estante y luego lo retorna.

Desarrolle un programa que:

- Construya dos estantes: uno para almacenar los libros sin clasificar dada su capacidad y la cantidad de libros recibidos y otro estante vacío, para almacenar los libros de la especialidad, a partir de una capacidad también dada.
- Realice el proceso de clasificación de cada uno de los libros que están sin clasificar.
- Muestre, la cantidad de libros que se movieron para el nuevo estante y la cantidad de libros que fueron regalados.

30. Pescador

Haciendo uso de la clase Pescador que se describe a continuación:

Pescador	
<u>Características y Funcionalidades</u>	<u>Descripción</u>
- <u>nombre</u> : String	<u>Nombre del pescador.</u>
- <u>capacidadMorrall</u> : double	<u>Cantidad de libras que caben en el morral del pescador.</u>
+ <u>Pescador(nombre: String, capacidadMorrall: double)</u>	<u>Construye un Pescador con un nombre y la capacidad de su morral en libras. El morral del pescador inicialmente está vacío.</u>
+ <u>Pescar()</u> : int	<u>Retorna un valor entero en dependencia del tipo de pescado que pescó el pescador. Retorna 1 si es Aguja, 2 si es Sábalo y 3 si es Robalo.</u>
+ <u>PuedeColocarEnMorrall(tipoPez: int): boolean</u>	<u>Retorna true si el tipo de pez pasado como parámetro (1, 2 ó 3) cabe en el morral del pescador. Retorna false en otro caso. Teniendo en cuenta que un pez aguja pesa 50 libras, un pez Sábalo pesa 20 libras y un pez Robalo pesa 15 libras.</u>
+ <u>ColocarEnMorrall(tipoPez: int): void</u>	<u>Coloca en el morral un tipo de pez pasado por parámetro(1, 2 ó 3). Es decir, decrementa la cantidad de libras que caben en el morral del pescador en el peso correspondiente al tipo de pez pasado por parámetro. Teniendo en cuenta que un pez aguja pesa 50 libras, un pez Sábalo pesa 20 libras y un pez Robalo pesa 15 libras.</u>
+ <u>MorrallLleno(): boolean</u>	<u>Retorna true si el morral está lleno y false en otro caso.</u>
+ <u>Puntuación(tipoPez: int): int</u>	<u>Retorna la cantidad de puntos que se le otorga al pescador por pescar un tipo de pez (1, 2 ó 3) pasado por parámetro. Teniendo en cuenta que por cada pez aguja se otorgan 5 puntos, por cada pez Sábalo se otorgan 4 puntos y por cada pez Robalo se otorgan 3 puntos.</u>

Desarrolle un programa que:

- Construya dos pescadores, dado el nombre y la capacidad del morral de cada uno de ellos.
- Haga pescar a ambos pescadores hasta que hayan pescado una cantidad de peces entrada por el usuario o que a alguno de los dos se le llene el morral.
- Finalmente, muestre el nombre del pescador que más puntos logró acumular.

31. Aeropuerto

En una terminal de carga se desea llevar el control de las valijas que se transportan en los aviones de pasajeros. Para desarrollar la aplicación que cubra este negocio se ha identificado la siguiente clase CAvionPasajeros con las funcionalidades que se describen a continuación.

CAvionPasajeros	
marca: String	Atributo que almacena la marca del fabricante.
cantidadMaxPasajeros : int	Atributo que almacena la cantidad máxima de pasajero.
capacidadMaxCarga : double	Atributo que almacena la capacidad máxima de carga.
cantidadRealPasajeros : int	Atributo que almacena la cantidad real de pasajeros.
capacidadRealCarga : double	Atributo que almacena la capacidad real de carga.
CAvionPasajero(marca : String, cantidadMaxPasajeros: int, capacidadMaxCarga : double)	Constructor de la clase CAvionPasajero donde inicializa los campos de la marca del fabricante, la cantidad máxima de pasajeros y la capacidad de carga.
CAvionPasajero(marca : String, cantidadMaxPasajeros: int, capacidadMaxCarga : double, cantidadRealPasajeros : int, capacidadRealCarga : double)	Constructor de la clase CAvionPasajero donde inicializa los campos de la marca del fabricante, la cantidad máxima de pasajeros, capacidad de carga, cantidad real de pasajeros y capacidad real de carga.
PuedeCargar (pesoCarga : double) : boolean	Método que retorna TRUE si el avión puede cargar el peso de la valija en el compartimiento que se está pasando por parámetro. En caso contrario retorna FALSE.
isDisponibilidadAsiento() : boolean	Método que retorna TRUE si existen asientos vacíos. En caso contrario retorna FALSE.
Cargar(pesoCarga : double): void	Método que carga el avión dado un peso de la valija y el identificador del compartimiento donde se puede cargar.
AdicionarPasajero(): void	Método que incrementa en 1 la cantidad de pasajeros dentro del avión.
ObtenerCarga() : double	Método que retorna el peso de la valija dado el identificador del compartimiento.
ObtenerPasajero() int	Método que retorna 1 pasajero. Además de decrementar la cantidad real de pasajeros 1.
ExisteCarga(): boolean	Método que retorna TRUE si todavía el avión tiene carga. En caso contrario retorna FALSE.
ExistePasajeros() : boolean	Método que retorna TRUE si todavía tiene pasajeros. En caso contrario retorna FALSE.

Se desea un programa que permita:

- Construya un objeto de la clase CAvionPasajeros donde se le pasen los siguientes valores, marca = “Boeing”, cantidad máxima de pasajero = 250, capacidad máxima de carga = 10000 kg, cantidad real de pasajero = 245 y cantidad real de carga = 8575 kg.
- En la pista donde se encuentra el avión tipo Boeing que se creó pasó un problema. El avión tiene un desperfecto y no puede volar y la aerolínea ha decidido realizar un transbordo de la carga y de pasajeros. En este momento la aerolínea no cuenta con un avión de estas características, por lo tanto tiene que utilizar dos aviones un 150 y 100 de cantidad de pasajero y 6000 kg y 5000 kg de capacidad de carga respectivamente.
- Construya dos objetos de tipo CAvionPasajeros.
- Implemente un algoritmo que realice el proceso de transbordo teniendo como restricción la siguiente premisa. Hasta que no se llene el primer avión tanto como de pasajeros y de carga, no se debe pasar a llenar el otro avión. Usted decide cual es el primer avión a cargar.
- Al finalizar el transbordo se necesita mostrar cuántos pasajeros y qué peso de carga lleva cada avión.

32. Cámara Fotográfica

Una cámara digital de fotos se compone de una memoria donde almacena las fotos digitales a medida que éstas se van tomando. Si la memoria está llena y se quiere seguir tomando fotos, entonces hay que vaciar la memoria. El espacio en memoria que ocupa una foto está en dependencia de su calidad (a mayor calidad, mayor espacio en memoria requiere). El diseño y la especificación de la clase es el siguiente.

CamaraFotografica	
Atributos y Métodos	Descripción
- numeroSerie: String	Atributo que almacena el número de serie de la cámara.
- capacidadMB : int	Atributo que almacena la capacidad de la memoria de la cámara.
- capacidadOcupada : int	Atributo que almacena la capacidad ocupada por fotos almacenadas en la memoria de la cámara.
+ CamaraFotografica(numeroSerie : String, capacidadMB : int)	Constructor de la clase CamaraFotografica donde inicializa los campos del número de serie y la capacidad máxima de memoria. Inicialmente no hay fotos almacenadas en la memoria.
+ TomarFoto (tipoFoto : int) : void	Método que toma una foto de un tipo determinado introducido por el usuario. Este parámetro que recibe tendría un valor de 1, 2, 3 según el tipo de foto. La cantidad de memoria que requiere este tipo de foto debe ser adicionada a la cantidad de memoria ocupada.
+ PuedeTomarFoto(tipoFoto : int): boolean	Método que recibe como parámetro el tipo de una foto (valores 1, 2, 3). Retorna TRUE si la cámara puede tomar una foto de ese tipo, es decir, si la cantidad de MB en uso junto con la cantidad de MB que requiere el tipo de foto pasado por parámetro no excede la capacidad de MB de la cámara. Retorna en caso contrario.
+ CapacidadMB() : int	Método que retorna la capacidad de memoria de la cámara.
+ NumeroSerie() : String	Método que retorna el número de serie de la cámara.
+ Vacía(): boolean	Método que retorna TRUE si la memoria de la cámara esta vacía. En caso contrario retorna FALSE.
+ Llena(): boolean	Método que retorna TRUE si la memoria se encuentra llena. En caso contrario retorna FALSE.
+ AdicionarFoto(tipoFoto : int) void	Método que permite adicionar una foto a la memoria de la cámara. Esta función no utiliza el método TomarFoto.
+ ExtraerFoto(): int	Método que retorna el tamaño de la primera foto eliminándola de la memoria de la cámara. Esta función libera los MB utilizados por la foto extraída.
+ toString() : String	Método que retorna las especificaciones de la cámara.

Se quiere modelar una cámara digital con la que se puede tomar 3 tipos de fotos:

- Tipo 1: Foto de calidad baja. Una foto de este tipo requiere 1 MB de memoria para almacenarse.
- Tipo 2: Foto de calidad media. Una foto de este tipo requiere 3 MB de memoria para almacenarse.
- Tipo 3: Foto de calidad alta. Una foto de este tipo requiere 5 MB de memoria para almacenarse.

Desarrolle un programa que:

- Construya un objeto de tipo CamaraFotografica donde los datos sean introducido por el usuario. Debes permitirle al usuario que seleccione el tipo de foto. Además tome foto hasta que se llene la memoria. En caso de que no se pueda tomar fotos de tipo 3 porque no hay espacio, verificar si hay espacio para una de tipo 2, sino verificar si hay espacio para una de tipo 1; en cual quiera de los caso que exista espacio tomar la foto.
- Construya dos cámaras una con 128 MB y otra con 1024 MB de memoria

respectivamente. El usuario quiere pasar las fotos de tipo 5 y 3 para la cámara que tiene mayor espacio en memoria y las de tipo 1 para la cámara de menor memoria. Mientras las dos cámaras no estén llena y existe foto en la cámara creada en el inciso a) pasar la foto.

- Muestre en pantalla el número de serie de la cámara y la cantidad de fotos que tienen las dos

cámaras creadas. En caso de que la 1ra cámara creada no esté vacía mostrar en pantalla su especificación.

33. Terminal de ómnibus

En una terminal se lleva el control de los ómnibus, los pasajeros que viaja y precios de los pasajes. De la terminal se conoce su nombre, la cantidad de ómnibus disponibles para el diario y cantidad de rutas o itinerarios. Esta terminal solo utiliza guaguas YUTON con una capacidad de 45 personas. Está penalizado que en los ómnibus existan personas de pie durante el viaje. Para desarrollar la aplicación que cubra este negocio se ha identificado la siguiente clase CTerminalPasajeros con las funcionalidades que se describen a continuación.

CTerminalPasajeros	
Atributos y Métodos	Descripción
nombre: String	Atributo que almacena el nombre de la terminal de pasajeros.
cantidadOmnibus : int	Atributo que almacena la cantidad de ómnibus que tiene disponibles la terminal de pasajeros.
cantidadRutas : int	Atributo que almacena la cantidad de rutas que tiene que cubrir la terminal de pasajeros.
CTerminalPasajero(nombre : String, cantidadOmnibus : int, cantidadRutas : int)	Constructor de la clase CTerminalPasajero donde inicializa los campos del nombre de la terminal de pasajeros, la cantidad ómnibus disponibles y la cantidad de rutas a cubrir.
PasajerosEnCola (numeroRuta : int) : int	Método que dado el parámetro que identifica la ruta, retorna la cantidad de pasajeros que se encuentran en la cola.
ObtenerOmnibusDisponible(): String	Método que retorna el identificador del ómnibus (Chapa) que está disponible para atender la ruta y se afecta decrementando en 1 el atributo de la cantidad de ómnibus disponibles siempre y cuando exista más de 1 ómnibus. En caso que sea 0 indica que no hay disponibilidad de ómnibus.
isDisponibilidadOmnibus(): boolean	Método que retorna TRUE si existe disponibilidad de ómnibus para atender una ruta. En caso contrario retorna FALSE.
PrecioPasaje(numeroRuta : int) : double	Método que dado el número de la ruta retorna el precio del pasaje.
ObtenerNumeroRuta() : int	Método que retorna el identificador de la ruta a atender en el momento y se afecta decrementando en 1 el atributo de cantidad de rutas a atender y cuando exista más de 1 ruta por atender. En caso que sea 0 indica que ya se atendieron todas las rutas planificadas para el día.

Desarrolle un programa que:

- Construya un objeto de la clase CTerminalPasajeros donde los datos sean introducido por el usuario.
- Simule lo que la terminal de pasajero desarrolla durante un día. La misma comienza a las 5:00 AM y termina de atender todas las rutas 24:00 (Hora Militar). El objetivo de la terminal es atender todas las rutas y llevar el control los ómnibus.

Descripción del proceso de llenar ómnibus: De forma diaria la terminal de ómnibus atiende a todas las rutas siempre y cuando exista disponibilidad de ómnibus. Si existe disponibilidad, se selecciona la ruta y el ómnibus y se procede a atender la cola de pasajeros. A medida que cada pasajero vaya abordar al ómnibus este debe pagar en dinero en efectivo el precio del pasaje. Cada vez que se atiende un ómnibus se debe mostrar el identificador del ómnibus, la cantidad de pasajeros a bordo y precio total. Al

finalizar la jornada se debe mostrar cuánto se recaudó, cuantas rutas fueron atendidas y cuantos pasajeros fueron transportados por la terminal.

34. Artistas aficionados

En vista al festival de artistas aficionados de la facultad el vicedecanato de extensión solicitó una aplicación para determinar la disponibilidad de los implicados teniendo en cuenta sus resultados académicos. Si se tiene la siguiente descripción de clase:

<u>ArtistaAficionado</u>	
Funcionalidad	Descripción
- nombre: <u>String</u>	Almacena el nombre del participante
- annoAcad: <u>int</u>	Almacena el año académico del participante
- evaluacCorte: <u>char</u>	Almacena la evaluación obtenida en el último corte académico
- modalidad: <u>String</u>	Almacena la modalidad artística en la que participará
+ <u>ArtistaAficionado(nombre: String, anno: int, eval: char, modal: String)</u>	Crea un nuevo AA con los datos entrados por parámetros
+ <u>PuedeParticipar(): boolean</u>	Retorna verdadero si puede participar en el Festival y falso en caso contrario. Vea la descripción anexa para las orientaciones.
+ <u>Nombre(): String</u>	Retorna el nombre del participante
+ <u>AnnoAcademico(): int</u>	Retorna el año académico del participante
+ <u>NotaCorte(): char</u>	Retorna la nota obtenida en el último corte evaluativo
+ <u>Modalidad(): String</u>	Retorna la modalidad en la que se presentará el participante
+ <u>toString(): String</u>	Retorna una cadena de caracteres que representa la entrada en la cartelera del Participante.

Desarrolle un programa que solicite al usuario la cantidad de participantes, los datos de cada participante y cree la cartelera del Festival de Aristas Aficionados de la facultad 6.

- La cartelera se forma con los datos de los Estudiantes que pueden participar. Cada entrada tiene la forma “Nombre del participante – Año académico – Modalidad”.
- Un estudiante puede participar cuando:
 - Si es de 5to año su calificación debe ser de B.
 - La modalidad de teatro requiere una gran cantidad de horas de ensayo, por lo que no aceptará presentar ningún estudiante con nota R.
 - Música solo acepta estudiantes con R si estos cursan el 3er año o menor.
 - Plástica acepta a todos los participantes.
 - Danza acepta a estudiantes de 3er año en adelante con nota de aprobado.

35. Licencia de conducción

Con vista a cumplir con las nuevas leyes de vialidad y la ley 60 del tránsito, la PNR quiere desarrollar una aplicación para determinar el listado de los infractores que pasan un día X por el punto de control de la carretera a San Antonio. Un ingeniero de software proveyó la siguiente descripción de la clase LicenciaConducción, en la cual se almacenan los datos del conductor.

<u>LicenciaConduccion</u>	
Funcionalidad	Descripción
- <u>nombre: String</u>	Almacena el nombre del conductor
- <u>puntosInfractor: int</u>	Almacena los puntos de infracción que ha tenido en el año.
- <u>categoria: char</u>	Almacena la categoría de la licencia
- <u>habilitada: boolean</u>	Indica si la licencia está activa.
+ <u>LicenciaConduccion(nomb re: String, cat: char)</u>	Crea una nueva licencia con los datos entrados por parámetros.
+ <u>Nombre(): String</u>	Retorna el nombre del conductor
+ <u>PuntosMulta(): int</u>	Retorna los puntos que ha acumulado como infractor
+ <u>Categoria(): char</u>	Retorna la categoría de la licencia de conducción
+ <u>EstaHabilitada(): boolean</u>	Retorna true si la licencia actual está habilitada y false en caso contrario.
+ <u>toString(): String</u>	Retorna una cadena de caracteres de la forma "Nombre conductor - categoría - habilitada/deshabilitada"
+ <u>Multar(gravedad: char)</u>	Multa al conductor. Ver descripción del proceso adjunta al ejercicio.

Asuma que cada conductor pasa 4 veces por el punto de control (ida y vuelta 2 veces al día), cada vez que pasa se le toma la velocidad con la que pasó a través del punto de control. Si dicha velocidad excede los 40 Km/h el conductor es multado.

Para aplicar la multa se le adicionan a los puntos de infractor la cantidad acorde a la categoría y la gravedad de la infracción.

Una multa leve se impone a un infractor que pase a una velocidad menor a 50 km/h.

Una multa media se impone a un infractor que pase a una velocidad menor de 70 km/h.

Una multa grave se impone a los infractores que pasen a una velocidad mayor o igual a 70 km/h.

Categoría A (Ómnibus):

Leve – 3 ptos, Medio – 6 ptos, Grave – 9 ptos

Categoría B (Camión):

Leve – 2 ptos, Medio – 3 ptos, Grave – 4 ptos

Categoría C (Moto) y D (Automóvil):

Leve – 1 pto, Medio – 2 ptos, Grave – 3 ptos

Si los puntos de infractor pasa de un valor determinado por la categoría de la licencia, la misma se deshabilita.

- Categoría A: 28 puntos
- Categoría B: 15 puntos
- Categorías C y D: 10 puntos

Desarrolle un programa que solicite los datos del conductor y los valores de velocidad de las 4 veces que pasó por el punto de control. Esta solicitud se efectuará hasta que el nombre del conductor sea “fin”. El programa debe mostrar en pantalla el listado de los infractores y el estado de sus licencias de conducción.